DEEPDIVES

The impact of AI on software engineers in 2026: key trends

Our AI tooling survey finds concerns about mounting AI costs, more engineers hitting usage limits, and AI tools having uneven effects upon different types of engineers

GERGELY OROSZ AND ELIN NILSSON

APR 14, 2026 · PAID

334 8 24 Share

Recently, we ran a survey asking readers of The Pragmatic Engineer how you use AI tools, which tools you use, what does and doesn't work, and what it's like working with AI, in general.

For today's issue, we've dug into your 900+ responses to look for trends in AI tool usage among software engineers and engineering leaders. This article surfaces insights that are less about specific tools, and more about the effect these tools have on tech professionals. We cover:

1. **Costs.** Unsurprisingly, companies pay for most tool usage, and those responsible for budgets are increasingly nervous that AI-related costs are headed only one way: up.
2. **Usage limits.** Around 30% of respondents say they have hit limits. Switching tools, upgrading plans, or moving over to API pricing are common responses.
3. **Impact on "Builders."** Folks who make larger code changes and do "quality-of-life" work are builders, and they're also dealing with more AI slop. Some also grapple with a loss of professional identity.
4. **AI tools speed up "Shippers."** Engineers who focus more on getting things done are the most positive about AI tools. But they also add tech debt faster and might build the wrong things.
5. **"Coasters:" learning faster while generating AI slop.** Less adept engineers can uplevel faster with AI, but they generate a lot of "AI slop" while doing so, which frustrates builders.
6. **Changing software engineer & engineering manager (EM) roles.** Engineers have to orchestrate and context switch more often, while engineering managers

can be more hands on. It's interesting to see the engineer and manager roles becoming more similar.

7. **Other impacts on the craft.** We're going from "how" to build to "what" to build, solo devs are seeing improved results, workloads are increasing with AI tools, and more.

We previously published a detailed summary of the survey which focused on AI tooling for software engineers, covering the most-used AI tools, trends, AI agent usage, company size and usage, and tools that engineers love.

1. Costs

Concern about the cost of AI tools is a trend throughout the survey, with around 15% of respondents mentioning it in some way.

Tech companies foot the bill for the majority of spending on AI tools. More respondents say their employers pay for AI coding tools than those who say they pay themselves, and predictably, employers fund more expensive packages than what individuals buy personally.

Companies commonly pay for "max" plans with the likes of Claude Code, Cursor, and Codex (around $100-200/month per engineer), although some companies' budgets only stretch to $20/month per engineer – around the price point of GitHub Copilot, and the cheapest Claude or ChatGPT subscriptions.

The most-mentioned AI tool spending patterns:

- **When companies pay:** ~$200/month plans. Many have enterprise subscriptions, sometimes with subsidies and vendor lock-in. Some companies allow usage-based coverage on top of monthly plans.
- **When personally paying for tools:** ~$20/month or free tiers. This can stack up across different tools. Around 5% of respondents have separate work and personal subscriptions, and free tier usage is widespread for personal use.

For now, companies seem to be in the experimentation phase with AI tools, and several respondents say that they believe their companies have unsustainable AI-tooling budgets. This is likely because businesses are figuring out the best way of

leveraging the tools, and the message to engineers at such places is to not worry about price and usage while that unfolds. A CTO at a small, US-based company shares:

> "Right now, we're not sweating the costs because we're trying to evolve best practices for the tools, but that has resulted in some devs really blowing through budget, so we may start instituting caps on spending."

Breaking the budget

At small and mid-sized companies, leadership teams seem more comfortable about going over budget, than engineers running out of budget. There are more accounts from C-level folks and founders about racking up large bills than there are from engineers. A CPTO (Chief Product and Technology Officer) at a mid-sized company:

> "I ran up several monthly bills of $600 with Cursor. We have the dev team subscribed to ~$100/month plans. We're now in the process of moving the rest of the team to Claude Code, as we can get more resources for around $100/month in cost."

Top spenders can be allocated higher budgets. A number of tech businesses have separate, larger budgets for their heaviest AI users. A senior C++ engineer working in the video game industry says:

> "I've become my team's AI champion. In theory, my limits are higher than normal, but I keep myself limited to what others can use, so I can show them useful things they can do."

UK and EU companies worry more about budgets than US-based ones. Most responses which mention finance teams pushing back against spending even $30-50/month per engineer on AI tools are based in the UK and EU. One amusing example is a 10-person, seed-stage startup where the CEO questioned why they were paying as much as £25/month per engineer for one of the cheapest AI tools around.

In general, it feels like European companies want to see clear value-add in order to justify an increase in tooling spend, whereas US companies are more comfortable with investing first and measuring impact later. At present, the impact of these tools is hard to quantify.

A niche approach is that of AI teams educating devs to use cheaper models. Some European companies go as far as offering education to new joiners on using cheaper models. From an AI Enablement Lead at a 1,000+ person, digital transformation company:

> "Within our organisation, we've had incidents where our Claude users have overshot their limits. We're now attempting to educate devs in knowing the difference between different models (knowing when to use Claude Sonnet versus Claude Opus)."

Cost trajectory worries

The cost trajectory of AI tools is generally considered unsustainable in our survey. Devs using the tools heavily tend to hit usage limits, and their employers then have to pay more. At places with API-based pricing, usage is increasing. Those in leadership positions who are responsible for budgets are generally concerned about the direction of costs.

Subsidies are keeping things at bay – for now. A common enough pattern in our survey is of heavily-subsidized enterprise plans that come with vendor lock-in. Several responses raise concerns about what will happen when the subsidies run dry. Experienced engineering leaders recall that cloud providers also played the same game of subsidizing for a few years, then raising prices when a customer was fully "locked in."

The AI hype cycle is dampening awkward conversations about budgets at some places. A principal engineer at a Fintech tells us:

> "The AI hype has created a special, generous budget for AI tools, and there's no effective budget – yet!"

But some finance teams are getting grumpy. A CTO at a sports-tech company says:

> "It's hard to keep our CFO supportive about investing in these tools because the productivity benefits have proven difficult to conclusively prove. The point that resonated the most was the loss of value when people hit daily limits: having to stop work immediately! Surprisingly, our CFO is still pushing back, despite having experience of getting a lot of value through their own AI usage with their spreadsheets."

Most survey respondents think the price of AI tools will have to rise. If that happens, it would cause problems at several companies – particularly those in Europe:

> "I cannot see how the spend on AI tools is fiscally sustainable in its current form; Max 100 with Claude Code is $100 a month. A single small task powered by Kimi K2.5 using OpenCode is $5, mostly in input cost. If we assume that the third party inference providers are doing so at a sustainable price, the much more expensive Opus model cannot be sustainable, never mind profitable at these plan costs." — *Founder at a seed-stage company, Europe.*
>
> "From the economic perspective, at some point, these companies will need more funding or profit, I'm curious how much it costs them to have a proper agent, and still become profitable. It feels slow when you run out of credits when working on repetitive tasks." — *Principal Software Engineer at a seed-stage company, Europe*

2. Usage limits

Another major trend in our survey results is the topic of usage limits:

- **Hitting limits: ~30%** of respondents. Running out of tokens or hitting reset limits is frustrating and disruptive, especially when you're working on a task or are in a flow state. The majority of respondents who complain about hitting limits are on cheaper plans (typically $20/month.) But this issue is also mentioned about higher subscription levels.
- **Under the limit: ~20%** of respondents. Avoiding usage caps generally correlates with being on more expensive plans with higher limits, or in roles with enough non-coding work for it not to matter, or when devs do enough work "manually" for AI usage to not be an issue.

Why users of AI tools hit limits

Common reasons cited in the survey:

Being a new AI user or a power user. These are two distinct groups, but an engineering manager at a mid-sized company in Canada says that each one similarly blows through token limits for different reasons:

> "We're mindful of trying to manage costs by setting AI spend limits across the org. We have two subsets of users at odds with each other:

> 1. Individuals who are still learning and blow through their credits at an inordinate rate, forcing us to keep limits low.
> 2. Power users who hit the limit through regular use and apply pressure to raise the limit.
>
> It's a tough balance."

Using Opus for all work. A few engineers mention being careful about how they use Opus because it previously ate up their token budgets. Here's a software engineer at a mid-sized company in Europe:

> "I made the mistake of using Opus in the past and burning through budgets quickly. Now, my routine is to start in 'plan' mode with Opus. I then paste the acceptance criteria and description of the issue and let the plan mode figure it out. I then switch to Composer or Sonnet and have the agent take over from there."

Mistakes that eat up tokens are easy to make. These include starting on work or a problem from the wrong end, using AI directly for a task rather than opting for a simple script, trying some new tool or technique that ends up consuming tokens (OpenClaw and Ralph Loops are cited), and others.

What happens when the limit is hit?

Hitting the limit with an AI tool is inconvenient and happens to many developers, who take a variety of next steps:

Switch the model or tool. Around a quarter of respondents who hit limits mentioned switching. From a software engineer at working at Atlassian:

> "In my company, for Cursor and Windsurf we have monthly limits. Our internal coding tool (called codelassian) also has daily prompt and hourly token limits. When I hit a limit in one tool, I switch to the other."

Upgrade to a pricier plan. When it's an option, this is a no-brainer at most places, especially as the alternative would be devs twiddling their thumbs waiting for the limit to be reset. A senior engineering manager at a mid-sized company says:

> "In my team, we are regularly hitting session limits with Claude. We upgraded some teammates to the Max 20x plan – and on this plan we have not been hitting limits, so far."

Adopt API-based pricing. This is the easiest way to keep working without abandoning a task you're knees-deep in. A senior engineer at a large company says:

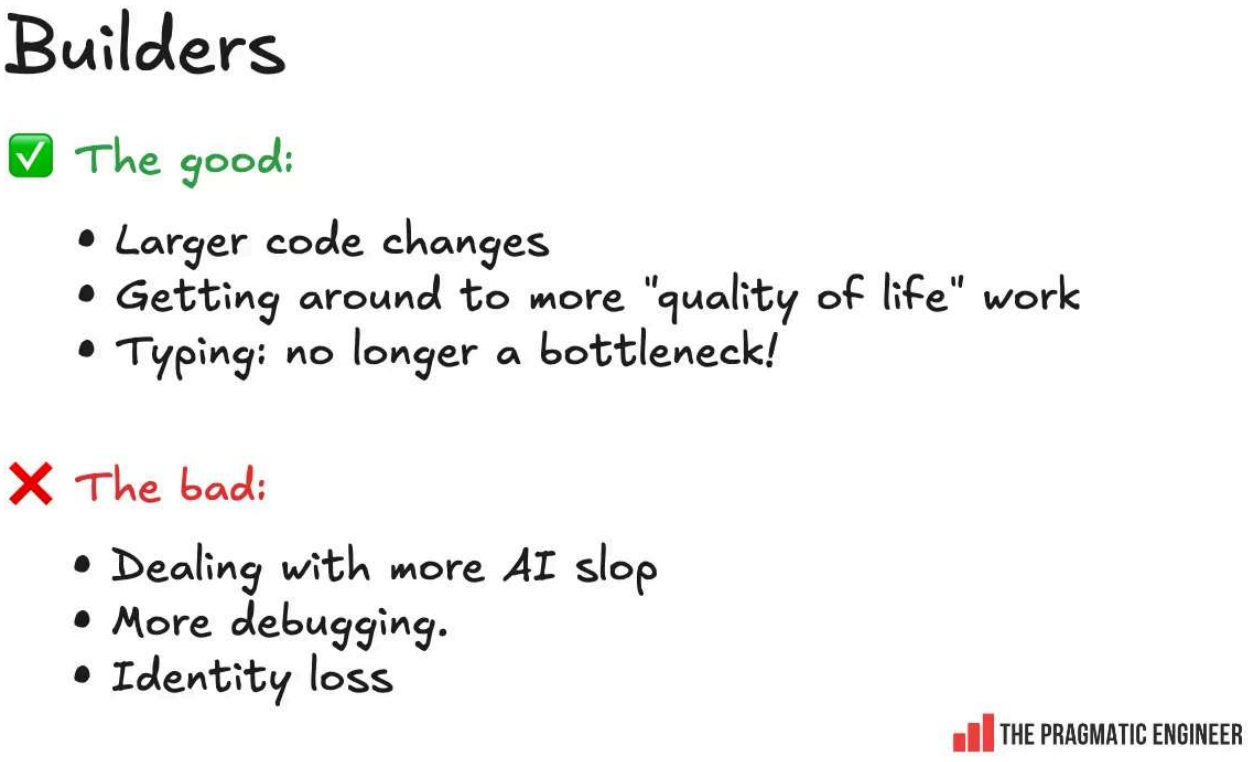
> "The company provides both the Claude and Copilot corporate offerings. When the limits are reached, I tend to use API keys that my teammates give me."

3. Impact on "builders"

We identified three different types of professional in the survey:

- **Builders**: those who care about quality, good architecture, following good coding practices, and who talk about the craft of software engineering, etc.
- **Shippers**: those who primarily focus on outcomes for a product, features, testing, and experimenting with users. A fair number of leaders, managers, and engineers who were more hands-off with coding before AI tools are in this category, as are product engineers.
- **Coasters**: engineers who are not considered particularly good or great engineers, but they get the work done. They often do this without much taste or concern for quality, and seem to be mostly coasting along and doing what they're told.

The overall consensus in our survey results is that AI will amplify and multiply tendencies and patterns that existed before, and the impact of the tools varies accordingly among users. Let's start with the impact we've observed upon builders in the responses:



The good and bad of AI tools, as shared by respondents in the Builder archetype

Builders say they get value from AI tools in the following areas:

Larger code changes. Builders generally find AI helpful for work like:

- Refactoring
- Migrations
- Improving test coverage
- Carrying out large codebase changes

All these are changes that are laborious, but not very challenging technically. They also require experience in knowing what you want to do and how to do it.

Accomplishing "quality of life" tasks. Builders mention that with AI tools, they get to fix and improve things like nagging bugs that otherwise wouldn't be "worth it" in time invested, but the barrier to entry is lower with AI.

A good example of this is in last week's podcast with David Heinemeier Hansson (DHH), the creator of Ruby on Rails, in which he revealed how one of their engineers optimized P1 – the fastest 1% of web requests:

> "One of our most agent accelerated people asked: "What about P1? What about the floor? Can we fix the floor?" He found that the floor [of request speed] was 4

> milliseconds.
>
> Well, 4 milliseconds can add up if you have a bunch of fast requests. So, he just said: "We're going to optimize P1. The fastest 1% of requests, we're going to make them even faster." He took it from 4 milliseconds to less than half a millisecond. He did this P1 project over a couple of days, like a side project.
>
> He had an intuition that there was something here. He let agents run with it. The work ended up being 12 pull requests, and about 2,500 lines of code changed.
>
> This is exactly why the explosion of the pie suddenly lets us look at problems we would never have contemplated looking at before."

Typing is no longer a bottleneck. Some builders report falling even more in love with coding with the help of AI and agents, since physically typing out code is no longer a bottleneck for them. They enjoy being able to prompt. From one "builder":

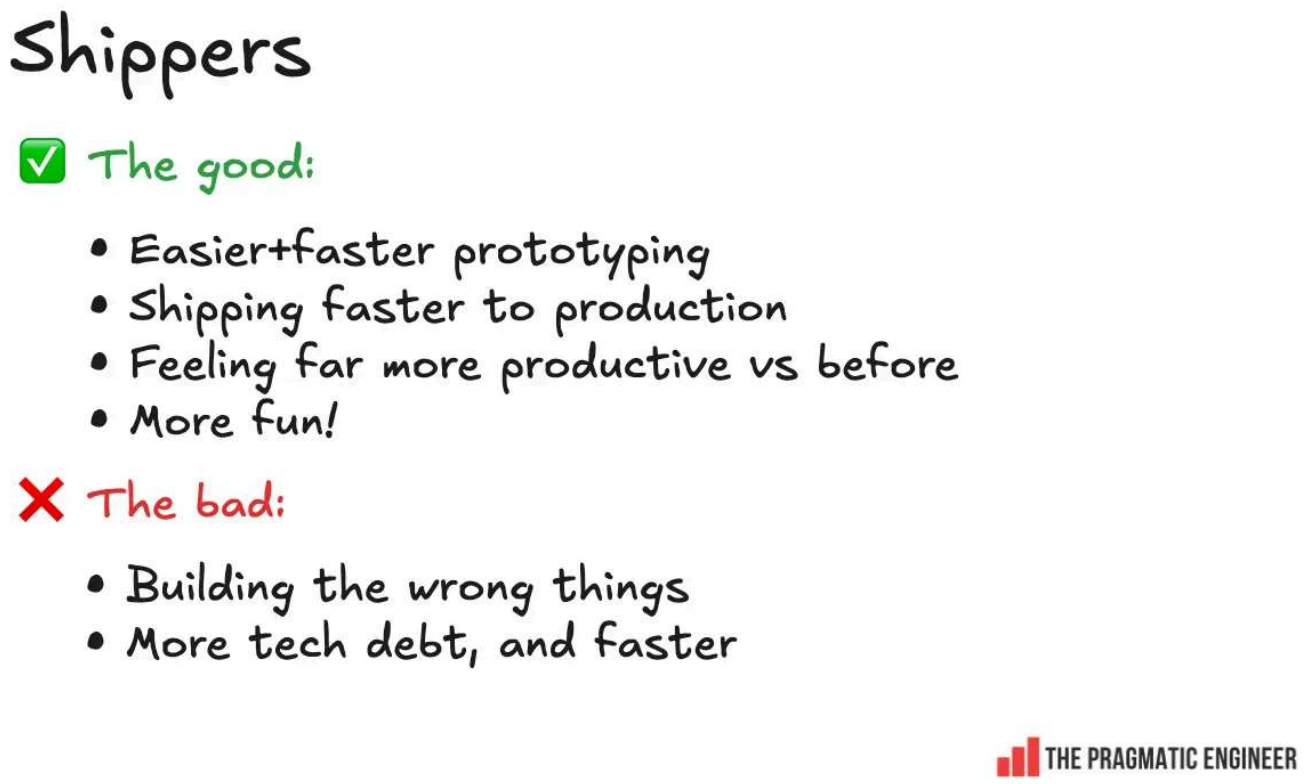
> "For someone who loves to build – but also values code quality, performance, reliability, and security – I ship a lot more quality code faster, if for no other reason than because the AI can read and write 100x faster than me. I get to stay at the conceptual level of shipping a product, and I can dive into debugging with the agent as needed. But if the agent has a good handle on the situation I can give it as much of the tedious parts as I wish." – *Staff Engineer, at a large tech company, US*

The negative sides of AI tools, as experienced by builders:

- **More AI slop.** Builders seem to be the most overwhelmed and derailed by reviewing a lot more AI-generated code. They can get frustrated with low-quality code shipped by colleagues which could be categorized as "AI slop."
- **More debugging.** AI-generated code introduces bugs and issues, and builders tend to spend the most time debugging and fixing those issues.
- **Identity loss.** Some builder-types report a sense of identity loss and even some grief. Much of this relates to no longer doing hands-on coding because they cannot justify it, since AI agents generate pretty decent code faster than someone can type it.

4. AI tools speed up "Shippers"

The “shipper” archetype thrives when they get things to production quickly. This group is by far the most enthusiastic about AI tools in survey responses. They are also the ones who praise – or hype up – the tools because of their personal experiences of shipping much faster with them.



Good and bad things about AI tools for shippers

The biggest upsides mentioned by shippers:

- **Easier, faster prototyping.** Building proof-of-concepts and prototypes is next-level with AI agents.
- **Shipping faster to production**. It’s not just prototypes; shippers can also churn out production work much faster now. And they do!
- **Feeling far more productive.** Thanks to faster iteration, they can do things faster. With agents running parallel, shippers can work on multiple things at once.
- **Having more fun.** For respondents who used to be less hands-on with coding, AI tools put the joy back into building. It’s now less about coding and more about orchestrating agents to turn high-level ideas into something workable.

Downsides mentioned by shippers:

- **Building the wrong things.** The responses indicate that some shippers may be working on a lot of half-formed ideas, like features that probably shouldn’t exist,

or apps with no real purpose. They're doing all this because it's just so easy to build.

- **More tech debt, faster:** more bugs, worse performance. Shippers tend to devalue correctness and code performance if the desired functionality is there. That's a recipe for tech debt.

Two illustrative quotes from shippers in our survey:

> "It makes programming fun again. Every person who says 'oh, but the craft of programming'. I was never like them. I saw programming as a way to accomplish an objective. Sure, there's beauty in the code. However, lots of apps that 'made' it are ugly as sin when you look at their codebase!" – *Engineering manager at a mid-sized company, US*
>
> "SWEs were never paid to write code nor learn codebases, you were paid to solve problems; writing code by hand was just a means to that end. We are paid to deliver value and build products people pay for." – *Staff Engineer at a small company, South America*

5. "Coasters:" learning faster but creating AI slop

A "coaster" is a professional who has a lot to gain from AI tools in terms of becoming more skilled and effective at their job, while also taking the load off of colleagues whom they might have turned to for support, pre-AI. Now, with these tools, their work can quickly be verified at the level of engineering practices.

Good and bad things about AI tools for coasters

The positives of AI for 'coasters':

- **A pathway to becoming a better engineer.** AI can empower coasters to become better engineers by acting as a mentor and giving them support that they didn't previously have.
- **More independence.** AI is a great learning and research tool that can reduce reliance upon colleagues for answers since the tool can access internal knowledge bases.
- **AI can protect against sloppy code.** One use case of AI is to validate and verify that engineering practices are followed, such as AI reviews to ensure there are tests for business logic changes. This automation protects against sloppy code.

Here's a quote from a lead AI engineer in India, who says they were not a great engineer, and that AI has helped them improve a lot:

> "I do not consider myself to be a great programmer, and I did not have access to that good of an education, either. I just learn a lot, learn fast, and adapt. So far, AI has helped me immensely. It has 3X'd my productivity in professional life and accelerated my professional growth by at least 2X."

There are downsides when coasters get going with AI:

- **AI slop.** Coasters seem to be the main producers of AI slop in codebases, according to respondents.
- **Not verifying AI's output.** Some devs try to get by doing the least possible amount of work. If a piece of AI output looks correct at first glance, they usually won't dig deeper.
- **One-shotting absolutely everything.** AI agents are increasingly good at one-shotting a ticket. Coasters will typically do this too: take a ticket, one-shot it with an AI tool, and then push it for review, without understanding the work and whether the tool made correct decisions.

A software architect at a smaller US company shares that coasters are creating AI slop for builders to deal with:

> "AI has amplified all the bad aspects of day-to-day interactions with colleagues: people who didn't care to do quality work now just sling slop at colleagues. People who did all the quality-review work are now just drowning because the review pipeline grows faster than they can process it."

6. Changing software engineer and EM roles

The jobs of software engineers and engineering managers are changing with the decline of manual coding.

Based on responses, the job of the software engineer is moving towards being an "agent orchestrator." Other mentions include feeling like the job is becoming more like that of an architect, a reviewer, or an "LLM manager". The focus is now on higher-level tasks like design and architecture of systems, planning, and breaking down work for agents, etc. Some people are very happy with this (shippers and coasters), while others (often builders) are less so.

Meanwhile, the engineering manager job is becoming more hands-on. EMs and engineering leads report spending more time hands-on with the code, improving their understanding of tech stacks and code bases, and being increasingly able to contribute technically because AI shrinks the ramp-up cost. From a director of engineering at a mid-sized company in the US:

> "I went from doing very little coding as a VP, to coding almost non-stop and joining meetings while giving my agent one last coding prompt. I can build prototypes,

> POCs, and production features in between meetings. It's fun, and it doesn't hurt that it's productive.
>
> I like to think my 20+ years of software development experience helps me better communicate and troubleshoot with the LLMs than a non-coder."

Conversely, engineering leaders being *too* hands-on can be disruptive. Whether this is a good or bad thing seems to depend on the individual and the team, as this account from a Senior SWE shows:

> "We recently had a director put up a 15,000-line pull request refactoring a critical feature with a lot of legacy code. He asked people what automation we needed to have in place to approve. Several staff engineers pushed back and said that without a proper planning and design phase, they could never approve a large-scale piece of AI-generated work like this. They also said that a task like this would need to be done iteratively across many PRs.
>
> The director was combative. The conversation ended with the director telling all staff+ engineers that at the next staff+ meeting, they would all be required to ask an AI tool to make a major improvement to our monolith. And that within an hour of the meeting, all of them would need to have a PR being submitted.
>
> **This director said that in the age of AI, a "wall of unreviewed, untrusted changes" will be unavoidable.** He said that Staff+ engineers will have to simply deal with this and have a process for how to approve those changes.
>
> I've never seen someone lose so much credibility so quickly. It was a major disruption across the org and I'm concerned that if this continues, our most valuable engineers will rage quit."

Context switching is increasing, which can be tougher to manage. On one hand, AI makes it easier to context switch and multitask by making it feel less expensive. On the other, more professionals feel increasingly drained by the amount of context switching they do with AI. Here's a lead engineer on this:

> "I am context switching way more because some work is done in parallel now. It's a bit tiring, but I get more work done."

Context switching is something that some engineers were able to avoid when being "in the zone" – until now, that is!

Code is no longer something you produce; it's something you consume. Several respondents mention that they got into software engineering because they love writing code. But the "writing code" part of the job is now shrinking. They refer to now "handcrafting" and "manual coding" at a time when the core of coding has shifted from writing it, to reviewing AI-generated output.

7. Other impacts on the craft

A selection of interesting observations from among the survey responses.

The mindset is shifting from "how" to build, to "what" to build. Responses mention needing to spend less time on implementation details and more on architecture, product thinking, and strategic decisions. Some folks describe this as a fundamental redefinition of their job.

> "My job is much more explicitly not to write code, but to solve problems and create tools. That's always kind of been the case, but now it's more clear. I think I'm able to tackle bigger, meatier problems than I had the time or bravery to do before". *Senior Engineer at a tiny/seed-stage company in the US*

Solo developers and small teams are seeing better results. Individuals and tiny teams report being able to accomplish work that previously required much larger groups.

> "I do the work of an entire team and work alone, which I prefer because of less overhead." *Engineering lead at a small company in the Africa & Middle East region.*

Workload is rising along with expectations. As AI makes it possible to move faster and ship more, the expectation also rises that these things are certainties. Individual contributors (ICs) in particular are reporting that expectations from management and leadership feel unrealistic. Some report that "doing a good job" doesn't seem to matter so much anymore, and that the quality of the work is increasingly overlooked as focus shifts to speed and output. Some expectations are also from external sources, as a solutions architect at Big Tech shares:

> "Unfortunately, due to high AI expectations, I get more tasks and now have even less time. For example, I am now writing the code on Java, JS (Node.js and React) etc. I am still trying to learn how to use them, and I'm reconsidering my career as I am stressed about news and layoffs."

In many cases, the pressures are internal. An engineering lead at a large company in Europe, says:

> "The worst impact is that I'm cramming even more things on my plate than usual because of AI. The work I'm doing nowadays in a week would have taken three times longer, five years ago."

Positive effects

Upsides mentioned most by respondents include:

Increased productivity. ~50% of respondents say they ship code between 2-5 times faster than before. Several factors contribute to the sense of being more productive:

- Less cognitive load, as AI absorbs the mental strain of syntax, boilerplate, wording, and routine problem-solving. This frees up mental energy for higher-level thinking.
- More confidence with unfamiliar tech stacks. Thanks to AI, engineers feel comfortable tackling languages, frameworks, and codebases they don't deeply understand.
- Reduced tooling friction: faster environment setup, easier to configure tools, easier to contribute to new codebases.
- Less time spent on tedious, repetitive work like boilerplate code, refactoring, documentation, and easy-to-automate tasks.
- Rapid prototyping and experimentation. AI is a great use case for these.

Better testing and documentation. AI makes traditionally-neglected tasks like tests, docs, and tech specs easy enough that devs now *actually* do them. Test coverage and documentation quality also go up. Two relevant quotes:

> "The best thing about AI coding assistants is having them write tests. Since working with Claude Code regularly, the code I have written has much much higher test coverage." – *Founding engineer at a seed-stage company, US*
>
> "Our documentation was never as great and accessible as it is now. Our internal tools finally have beautiful frontends as well." — *Engineering manager at a mid-sized company, Europe*

Quicker learning. Respondents report they learn new concepts, technologies, and codebases faster with AI as a learning partner. However, one engineer raises the question of whether we retain our new knowledge so well when the learning process is less strenuous than it was pre-AI.

Negative effects

Some commonly-cited negatives of AI in the survey:

Skill atrophy and tool dependence. This is the big negative mentioned by respondents. They report concerns about losing fundamental coding skills, becoming unable to work without AI, and of shallow understanding replacing deep expertise. A product manager summarizes it:

> "Without the exercise and stimulation of process and problem solving, and really getting entrenched, we are losing the ability to recall and exercise neural pathways. The brain works by strengthening and improving frequent areas of thought, and we are giving away those low-hanging thought areas to AI, which undercuts our ability to reach higher-level thinking."

Hype-related stress, rapid change, and fear of being left behind. Even for those who use AI tools daily, there's worry about being left behind. As a senior engineer in Asia puts it:

> "I hate all the hype. I don't know what I don't know: am I limited in my AI productivity gains by lack of knowledge? I really don't know. A lot of people make a lot of claims on Linkedin and other places, but are they actually building production applications with code, or just vibe-coded POCs with no users, and they [also] don't know what they don't know. I would love some content I can trust that shows really advanced AI workflows in production applications with real paying users."

Career anxiety and role uncertainty. Several devs admit to worries about job security, career relevance, and the future of software engineering as a profession. One director of engineering at a large US-based company tells us:

> "I have a lot more talks with my teams about what they're going to do once their career is dead. It's the subject of almost every 1:1 that I have across my entire org."

Increased context switching, cognitive load, overwhelm, burnout. AI agents enable parallel work, but increased context switching is cognitively draining, even when you're being productive. From a senior engineer in Australia:

> "I now work on two or more tasks at once in different codebases and jump between my AI agents' output all day long. I honestly don't know if it's made me more productive, but I'm definitely more tired. I don't know if I really am saving time and I don't trust the AI tools at all, so I triple check everything they do."

With Steve Yegge, we have covered The Dracula effect: the drain of using AI. He says:

> "There is something happening that we need to start talking about as a community. There's a vampiric effect with AI, where it gets you excited and you work really hard, and you're capturing a ton of value.
>
> I find myself napping during the day, and I'm talking to friends at startups and they're finding themselves napping during the day. We're starting to get tired and cranky."

Review fatigue and quality burden. Our survey finds reviewers – especially senior engineers – drowning in AI-generated PRs and spending most of their time checking output quality. One engineer cites getting 2,000+ lines of code pull requests to review, almost every day! It's humanly very difficult to review such long code changes, or rather, it's very hard to do well!

Takeaways

Thanks again to everyone who took part in the survey; we hope you find the results as interesting as we do. Reflecting on the findings, Elin told me:

> **"An amusing observation is how people feel super productive, but then complain about sloppy code everywhere – it seems to go hand-in-hand.**

As I went through responses, I saw similar patterns:

1. People who feel more productive than ever, outputting more code with higher quality and test coverage than ever, working on frameworks and codebases which they've never touched, and feeling really good and optimistic about AI, overall.

2. People who complain that colleagues have no idea what they're doing, don't understand languages and frameworks they've never used before, who produce code of low quality, and who ruin codebases with poor practices and terrible code, all because of AI.

I can't help wondering if these distinct responses could actually be two sides of the same coin in some cases.

Could concerns about costs be a proxy for worries about change? It's true that AI tools are more expensive than dev tools used to be. However, I still remember in the 2000s when Microsoft charged around $5,000-$8,000/developer per year for access to their Visual Studio tooling and MSDN documentation. That was $400-$700 per month, 20 years ago, and companies paid because having better tools meant devs were more productive.

We're seeing something similar now, except it's the first time that tool usage has no upper cap, when paying at the API-level or paying for tokens. And it's certainly a concern that costs are going up instead of down, when conventional wisdom states that compute gets cheaper over time.

For me, the most interesting finding in today's article is the differing benefits and drawbacks of AI for different users. "Coasters" are seen as folks who "let rip" with AI and generate lots of slop, which builders say they have to deal with. Meanwhile, shippers move fast, but accumulate lots of tech debt.

In our survey, most folks seem to be shippers, and a smaller number seem like builders, and it's they who complain about AI slop generated by coasters. In that case, it's no wonder that tech debt is rising at many companies!

For more findings from our survey, check out the accompanying deepdive, [AI tooling for software engineers in 2026.]

334 Likes · 24 Restacks

A guest post by

Elin Nilsson

Hej, I'm Elin! 👋 I do research for The Pragmatic Engineer, various naturey and crafty activities, and think random thoughts about

Subscribe to Elin

life, the universe and everything ✨

Discussion about this post

Comments Restacks

Write a comment...

Sri 15 kwi

Liked by Gergely Orosz

I'm not a fan of the "there is so much AI slop" narrative. If a colleague is shipping "AI slop," that's not an AI problem, it's an engineering discipline problem. The same person would be copy-pasting from Stack Overflow without understanding it if AI didn't exist. The tool isn't the issue. There are processes to catch this: PR review standards that hold code to the same bar regardless of how it was written, automated quality gates (coverage thresholds, static analysis, linting), spec-driven development where the design is agreed before code is generated, and pair programming where senior engineers can coach the workflow. If those processes exist and someone is still shipping low-quality code, that's a skill gap that needs addressing, not a reason to blame the tooling.

LIKE (11) REPLY SHARE

1 reply

Konstantin 15 kwi *Edited*

I wonder if there is a meaningful difference between "Coasters" and "Shippers"? Or these are largely the same people, and the "Shipper" is just a polite way to address "Coaster"?

I haven't found meaningful differentiation between them in this article.

Is one's man "Shipper" another reviewer's "Coaster"?

LIKE (4) REPLY SHARE

1 reply

6 more comments...

© 2026 Gergely Orosz · Privacy · Terms · Collection notice
Substack is the home for great culture